AIMERZ.ai
Aim.Act.Achieve

NOTES

Java Object-Oriented Programming (OOP) - III

Constructors, Constructor Calling using this() method
Static Block
Non-Static Block (Java Block) & Order of Execution(Java)

Constructors, Constructor Calling Using this() Method

What is a Constructor?

Think of a constructor like a recipe for creating something new. When we make a new object, the constructor sets up all its initial properties.

A constructor is a special method that's automatically called when we create a new object. It's like the setup instructions for a new device. It initializes the object's properties and prepares it for use.

Types of Constructors

Default Constructor

It's like an empty recipe that Java creates automatically if you don't write any constructor.

Meaning: **If you don't explicitly define a constructor for a class, Java will create a default one for you.** This default constructor initializes the object's properties to their default values, like `null` for objects and `0` for numbers.

Example

```java
class Student {
    String name; // Default value null is assigned
    // Java creates invisible default constructor
    // Student() { }
}

Student student = new Student(); // Works even without writing constructor
```

No-Argument Constructor

In Java, when we create an object without specifying any arguments, the no-argument constructor is called.

If we don't define a constructor, Java will generate a default one that does nothing.

However, if we define a no-argument constructor, we can specify initial values for object properties.

```Java
public class Car {
       int speed;

    // No-Argument Constructor
    public Car() {
        speed = 0;
    }

    // Displaying Car details
    public void displayInfo() {
        System.out.println("Speed: " + speed);
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car(); // Calls the no-argument constructor
        myCar.displayInfo();
    }
}
```

Real-World Analogy of Custom No-Argument Constructor

Think of opening a **new email account**. When you create it, certain things are already set up for you by default, like a **welcome email** or **default folders** (Inbox, Sent, Trash, etc.). You didn't specify these features; they came as defaults. In programming, using a no-argument constructor with custom initializations is similar to getting a "starter setup" for objects.

Real-Time Usage – Setting Default Values for a Bank Account

Suppose you are creating a BankAccount class, and when a new account is created, you want to set a default balance and a default account type. The no-argument constructor can do this.

```Java
public class BankAccount {
    double balance;
    String accountType;

    // No-Argument Constructor
    public BankAccount() {
        balance = 100.00;          // Default balance
        accountType = "Savings";   // Default account type
    }

    // Displaying account details
    public void displayAccountInfo() {
        System.out.println("Balance: Rs " + balance);
        System.out.println("Account Type: " + accountType);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount newAccount = new BankAccount(); // Calls the no-argument
constructor
        newAccount.displayAccountInfo();
    }
}
```

Output

```Java
Balance: Rs 100.0
Account Type: Savings
```

Key Points about No-Argument Constructors

Purpose: To initialize an object with default values when no specific values are provided during object creation.

Syntax: A no-argument constructor has the same name as the class and does not include any parameters.

Automatic Creation: If you do not explicitly define any constructor in a class, Java automatically provides a default no-argument constructor.

Custom No-Argument Constructor: You can also define your own no-argument constructor to set custom default values.

Parameterized Constructor

A **Parameterized Constructor** in Java is a type of constructor that accepts one or more parameters. This allows us to pass specific values to initialize the object's attributes during the creation of the object, providing more control and flexibility in setting up objects.

Key Points about Parameterized Constructors

Purpose: To initialize an object with custom values provided as arguments.

Syntax: A parameterized constructor has the same name as the class and includes parameters inside the parentheses.

Flexibility: Allows different objects to have unique initial values based on arguments provided when the object is created.

No Default Constructor if Parameterized is Defined: If you define only a parameterized constructor, Java will not automatically create a no-argument constructor. You must define it explicitly if needed.

How a Parameterized Constructor Works

When you create an object and pass arguments in the parentheses, the parameterized constructor is called. The values passed are assigned to the

object's fields, setting the properties based on specific input. This is different from the no-argument constructor, which assigns default values.

Syntax Example

```Java
public class Car {
       int speed;

    // Parameterized Constructor
    public Car(int s) {
        speed = s;     // Sets speed to the value of parameter s
    }
}
```

Real-World Analogy

Imagine you are **ordering a custom cake** from a bakery. You specify the flavor, size, and decorations. The bakery creates the cake exactly how you requested it, unlike a standard cake that comes with default settings. Similarly, in programming, a parameterized constructor customizes an object based on values passed during creation.

Real-Time Usage – Setting Up a Bank Account with Specific Details

Suppose you are creating a BankAccount class, and when an account is created, you want to initialize it with a specific balance and account type. This is done using a parameterized constructor.

```Java
public class BankAccount {
    double balance;
    String accountType;

    // Parameterized Constructor
    public BankAccount(double b, String aType) {
        balance = b;
        accountType = aType;
    }

    // Displaying account details
    public void displayAccountInfo() {
```

```
        System.out.println("Balance: Rs " + balance);
        System.out.println("Account Type: " + accountType);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount myAccount = new BankAccount(5000.00, "Checking"); //
Calls the parameterized constructor
        myAccount.displayAccountInfo();
    }
}
```

Output

```
Java
Balance: RS 5000.0
Account Type: Checking
```

Constructor Calling and Constructor Overloading

Constructor overloading is when you have multiple constructors in the same class with different parameters (like different ingredients). It allows you to create objects in different ways based on the information you provide.

Example of Constructor Overloading

Think of it like making different types of sandwiches based on what you have available:

1. **Making a Simple Sandwich**: Just bread.
2. **Making a Sandwich with Filling**: Bread and some filling.

In programming, it looks like this:

```java
class Sandwich {
    String bread;
    String filling;

    // Constructor for a simple sandwich
    Sandwich(String breadType) {
        bread = breadType; // Set the type of bread
        filling = "None"; // No filling for a simple sandwich
    }

    // Constructor for a sandwich with filling
    Sandwich(String breadType, String fillingType) {
        bread = breadType; // Set the type of bread
        filling = fillingType; // Set the filling
    }
}

public class Main {
    public static void main(String[] args) {
        Sandwich simpleSandwich = new Sandwich("White Bread");
        Sandwich cheeseSandwich = new Sandwich("Whole Wheat", "Cheese");

        System.out.println("Simple sandwich with " + simpleSandwich.bread +
" and filling: " + simpleSandwich.filling);
        System.out.println("Cheese sandwich with " + cheeseSandwich.bread +
" and filling: " + cheeseSandwich.filling);
    }
}
```

Constructor Calling: Think of it as following a chain of recipes where one recipe builds on another. One constructor can call another to set up an object step by step.

Constructor Overloading: It's like having different ways to make a sandwich based on the ingredients you have. You can create an object using different constructors with varying parameters.

The this() Method

The `this()` method in Java is used inside a constructor to call another constructor in the same class. It's like saying, "First, do this other recipe (constructor), and then I will add more ingredients (properties)."

Real-world Example - Restaurant Order:

```java
class Order {
    String item;
    int quantity;
    boolean isPackaged;
    double price;

    // Basic order
    Order(String itemName, int itemQuantity) {
        item = itemName;
        quantity = itemQuantity;
        price = calculatePrice();
    }

    // Takeaway order
    Order(String itemName, int itemQuantity, boolean checkForPacking) {
        this(itemName, itemQuantity); // First create basic order
        isPackaged = checkForPacking;
        if (isPackaged) {
            this.price += 2.0; // Add packaging cost
        }
    }

    private double calculatePrice() {
        // Simple price calculation
        return quantity * 10.0;
    }

    public static void main(String[] args) {
        // Using the Order class
        Order dineIn = new Order("Pizza", 2);
        Order takeaway = new Order("Burger", 2, true);

        System.out.println(takeaway.price);

    }
}
```

Output

```
Java
Order Price is:: 22.0
```

Static Block

What is a Static Block?

A **static block** is a block of code that is executed **only once** when the class is loaded into memory.

Static blocks are used to **initialize static variables** or to run code that only needs to be executed once.

Key Points about Static Blocks

Runs Once: Runs automatically when the class is loaded, before any other code in the class.

Used for Initialization: Primarily used to initialize static variables or perform a setup that applies to the whole class.

Before Main Method: Static blocks are executed even before the main method, making them ideal for initial setup.

Real-World Analogy: Announcement

"Think of a welcome announcement made when students enter a classroom. This announcement only happens once at the start and applies to everyone in the room. Similarly, a static block runs once for the class and is not repeated."

Example 1: Setting Up a Greeting Message

Consider a scenario where we want to display a greeting message only once when a new user joins an app.

```java
class User {
    // Static variable for welcome message
    static String welcomeMessage;

    // Static block to initialize the welcome message
    static {
        welcomeMessage = "Welcome to the App!";
        System.out.println(welcomeMessage); // Displayed only once
    }

    // Instance variable for each user's name
    String name;

    // Constructor for User
    public User(String userName) {
        name = userName;
    }

    public void displayUser() {
        System.out.println("Hello, " + name);
    }
}

public class UserTest {
    public static void main(String[] args) {
        // Creating User objects
        User user1 = new User("RAMESH");
        User user2 = new User("SURESH");

        user1.displayUser();
        user2.displayUser();
    }
}
```

Explanation

The static block initializes the welcomeMessage variable with a greeting when the User class is loaded.

This message is displayed only once, even though two user objects are created (user1 and user2).

Output

```
Java
Welcome to the App!
Hello, RAMESH
Hello, SURESH
```

Example 2: Initializing Configuration for a Game

In a gaming scenario, imagine that certain settings (like difficulty level) should be initialized only once, at the start of the game.

```
Java
class GameConfig {
    // Static variable for game difficulty
    static String difficultyLevel;

    // Static block to set the difficulty level
    static {
        difficultyLevel = "Medium"; // Initial setup
        System.out.println("Game Configured with Difficulty: " +
difficultyLevel);
    }

    public void startGame() {
        System.out.println("Starting game on " + difficultyLevel + "
difficulty.");
    }
}

public class GameTest {
    public static void main(String[] args) {
        // Creating game objects
        GameConfig game1 = new GameConfig();
        GameConfig game2 = new GameConfig();

        game1.startGame();
        game2.startGame();
    }
}
```

Explanation

The static block sets the difficulty level to "Medium" only once when the GameConfig class is loaded.

No matter how many GameConfig objects are created, the static block does not execute again, keeping the difficulty consistent.

Output:

```
Java
Game Configured with Difficulty: Medium
Starting game on Medium difficulty.
Starting game on Medium difficulty.
```

Key factors of Static Block

Static Block: Executes only once when the class is loaded, ideal for initial setups or configurations.

Used for Initialization: It's commonly used to initialize static variables or execute setup code that applies to the whole class.

Non-Static Block in Java (Instance Initializer Block)

In Java, a **Non-Static Block** (also called an **Instance Initializer Block**) is a block of code enclosed in curly braces `{}` and is not preceded by any keywords like `static`. Non-static blocks run every time an object of the class is created. They are executed before the constructor of the class.

Key Points about Non-Static Blocks

1. **Executed for Every Object**: Non-static blocks run each time an object of the class is created.
2. **Order of Execution**: Non-static blocks execute before the constructor, no matter which constructor is called.
3. **Purpose**: They are typically used to initialize instance variables or run some code whenever an instance is created, regardless of which constructor is used.

How Non-Static Blocks Work

When we create an instance (object) of a class, the **Java Virtual Machine (JVM)** goes through three main steps:

1. **Memory Allocation**: Allocates memory for the new object.
2. **Initialization with Non-Static Blocks**: Executes any non-static blocks in the class.
3. **Constructor Execution**: Executes the constructor to finalize the object initialization.

Basic Syntax

```Java
class ClassName {
    {
        // Code inside non-static block
        // This code will execute every time an object of this class is
created
    }

    // Constructor(s)
    ClassName() {
        // Code inside the constructor
    }
}
```

Example

Let's consider a simple example to see how non-static blocks work:

```Java
class Car {
    String model;
    int speed;

    // Non-static block
    {
        System.out.println("Non-static block executed.");
        speed = 50; // default speed for every car
    }

    // Constructor
    Car(String carModel) {
```

```
        model = carModel;
        System.out.println("Constructor executed.");
    }

    void displayInfo() {
        System.out.println("Model: " + model + ", Speed: " + speed);
    }

    public static void main(String[] args) {
        Car car1 = new Car("Sedan");
        car1.displayInfo();

        Car car2 = new Car("SUV");
        car2.displayInfo();
    }
}
```

Explanation of Example:

1. **Non-static block**: This block runs before any constructor. Here, it sets a default speed of 50 for every Car.
2. **Constructor**: The constructor only sets the model of the car.
3. **Object Creation**: When a Car object is created, the non-static block runs first, followed by the constructor.

Output

```
Java
Non-static block executed.
Constructor executed.
Model: Sedan, Speed: 50
Non-static block executed.
Constructor executed.
Model: SUV, Speed: 50
```

What's Happening?

The non-static block runs **before** the constructor each time an object is created.

Each Car object (both car1 and car2) has a speed initialized to 50, even though this wasn't specified directly in the constructor. This demonstrates how a non-static block can ensure consistent values or actions across all objects, regardless of the constructor used.

Real-World Analogy

Imagine a **factory assembly line** for making custom cars. Each car goes through a **basic setup** stage where standard components are added, like wheels and seats, before it reaches a station where a unique feature (like color or model) is added based on customer specifications. The **basic setup** is like the non-static block that ensures every car has certain standard parts, while the **customization** is like the constructor that adds unique details.

Use Cases of Non-Static Blocks

Common Initialization for All Constructors: Non-static blocks are useful when you have multiple constructors but need to run some code in all of them. Instead of duplicating code in each constructor, you can place it in a non-static block.

```Java
class Book {
    String title;
    double price;

    // Non-static block
    {
        price = 9.99; // default price for every book
        System.out.println("Default price set in non-static block.");
    }

    Book(String title) {
        this.title = title;
    }

    Book(String title, double price) {
        this.title = title;
```

```
        this.price = price;
    }
}
```

Here, the non-static block sets a default price for all books, regardless of which constructor is called.

Example with Multiple Constructors

```Java
class Player {
    String name;
    int score;

    // Non-static block for initializing score
    {
        score = 10; // default score for every player
        System.out.println("Non-static block: Default score initialized.");
    }

    // Constructor with no parameters
    Player() {
        name = "Unknown";
        System.out.println("Constructor with no parameters called.");
    }

    // Constructor with one parameter
    Player(String name) {
        this.name = name;
        System.out.println("Constructor with name parameter called.");
    }

    void displayInfo() {
        System.out.println("Name: " + name + ", Score: " + score);
    }

    public static void main(String[] args) {
        Player player1 = new Player();
        player1.displayInfo();

        Player player2 = new Player("Alex");
        player2.displayInfo();
    }
```

```
}
```

Output

```Java
Non-static block: Default score initialized.
Constructor with no parameters called.
Name: Unknown, Score: 10

Non-static block: Default score initialized.
Constructor with name parameter called.
Name: Alex, Score: 10
```

Advantages and Disadvantages

Advantages:

- **Consistent Initialization**: Allows for consistent initialization across constructors without duplicating code.
- **Code Organization**: Keeps constructors clean, making code easier to read and maintain.

Disadvantages:

- **Order Dependency**: Since non-static blocks run before constructors, they might cause confusion if not used carefully.
- **Alternative Options**: Often, the same functionality can be achieved with a private method that both constructors call, making the code more explicit.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan